

GLM 5.3, Kimi K3, and DeepSeek V4 Pro as Independent Review-Council Seats

Diagnosing and Fixing Empty-Verdict Failures on a Provider-Pinned,
Body-Verifying Multi-Model Review Harness over OpenRouter
A Practitioner's Field Note, following The Intelligence Council

Adnan Tanveer (Addy)

Speedrun AI Labs

Sydney, Australia

speedrunlab.ai

August 21, 2026 | Paper Version 1.0, submitted to SSRN

NOTE ON TIME-SENSITIVITY: This note documents behaviour measured on August 21, 2026, against three specific model versions served through one model aggregator: GLM 5.3 (`z-ai/glm-5.3`), Kimi K3 (`moonshotai/kimi-k3`), and DeepSeek V4 Pro (`deepseek/deepseek-v4-pro`). Model defaults, provider routing, and aggregator behaviour all change on the order of weeks. The failure mechanisms and the configuration method described here are intended to be stable; the exact token ceilings, effort names, and provider pins are worked examples. Re-measure the reasoning-token spend of any reasoning-first model against your own payloads before trusting a ceiling.

Abstract

A multi-model review panel is only as good as the verdicts it actually collects. In production we found that three of our strongest independent seats returned *empty* verdicts under load: a 200 OK response whose body carried no review at all. The naive reading, that the models had failed, was wrong. Every case traced to the interaction between a reasoning-first model, whose hidden reasoning draws from the same output-token budget as its visible answer, and a harness whose defaults were tuned for non-reasoning models.

This note reports the diagnosis and fix, grounded in controlled experiments run the same day. We show that the widely-seen error string "Expecting value ... (char 7315)" is not a corruption offset but the byte length of an all-whitespace body, and that several distinct failures hide behind the same empty result. GLM 5.3 and Kimi K3 exhaust the token budget on hidden reasoning at their default effort; the fix is to cap reasoning effort. DeepSeek V4 Pro cannot be capped, and failed in three layers that we peeled back one at a time: a long silent reasoning window that a non-streamed request drops, a token ceiling that its reasoning alone exhausts, and, deepest and most instructive, a pinned backend provider that returned instant timeouts on the hard request while its published uptime read 99 percent. Its fix is streaming, a larger ceiling, and a health-ordered provider preference that routes around a degraded backend. We report honestly that our first DeepSeek fix looked complete and was not, and that only retesting under the real workload exposed the provider layer. We give the exact per-model configuration and the harness code that encodes it.

We make four contributions: (1) we characterise the empty-verdict failure and decode its diagnostic signature; (2) we separate its mechanisms, budget exhaustion, the silent-window drop, and a degraded backend provider, and show each with measured evidence; (3) we give a per-model setup table and a reference harness that any coding agent or review pipeline can adopt; (4) we report the review quality recovered, on real security and code-review tasks, once each seat is configured correctly. This is a practitioner experience report.

Keywords: multi-model review, reasoning models, LLM orchestration, model aggregators, reliability, agent harness, streaming

1. Introduction

Our prior note, *The Intelligence Council* [1], described a standing panel of independent frontier models convened to review a plan, a diff, or a decision before it is acted on. Its value is independence: different vendors and training corpora give genuinely separate reasoning, which a single model cannot supply when asked to check its own work. That note also flagged, in passing, a "reproducible configuration failure mode in which mandatory-reasoning models return empty verdicts when their token budget is set too low." This note is the full account of that failure, now that we have diagnosed it against three specific seats and fixed all three.

The practical problem is sharp. A council seat that returns an empty body does not announce itself as broken. It returns HTTP 200. If the harness counts a non-error response as a vote, an empty body is silently read as assent, and a nine-seat panel quietly becomes a seven-seat panel with two abstentions that nobody chose. On an autonomous platform where the council gates production changes, that is the difference between a real second opinion and a false one.

The models are not the problem. The three seats in this study are among the strongest independent reasoning models available at the time of writing. The problem is that a reasoning-first model has an operating envelope that a non-reasoning model does not, and a harness written for the older envelope drives the newer model straight into its failure region. This note is about that envelope, and how to configure for it.

2. Background: the shared token budget

A reasoning-first model produces two kinds of output: hidden reasoning tokens and visible content tokens. On the aggregator we use, both are drawn from the same `max_tokens` ceiling. If reasoning consumes the whole ceiling, zero tokens remain for the visible verdict, and the response terminates with an empty or whitespace-only body.

Whether this happens is governed by three model-specific facts: whether reasoning is mandatory, what the default reasoning effort is, and how many reasoning tokens that effort actually spends on a given payload. Table 1 records these for the three seats, pulled live on the measurement date. The two facts that matter most are in the last two columns: the default effort, and whether the model even offers a lower one.

Model (slug)	Reasoning	Default effort	Efforts offered	Ceiling (max completion)
GLM 5.3 (z-ai/glm-5.3)	Mandatory	max	max, high, low	131,072
Kimi K3 (moonshotai/kimi-k3)	On by default	max	max, high, low	(provider-set)
DeepSeek V4 Pro (deepseek/deepseek-v4-pro)	On by default	high	xhigh, high	393,216

Table 1: Reasoning configuration of the three seats, as reported by the aggregator on August 21, 2026. GLM 5.3 and Kimi K3 default to "max" effort and can be turned down; DeepSeek V4 Pro's lowest offered effort is "high", so it cannot be turned down below the level that causes the problem.

3. The two failure modes behind one empty body

3.1 The diagnostic signature: "char N" is a length, not an offset

The failure first presented as a parser error, logged across several convenings as `Expecting value: line 1333 column 1 (char 7315)` and close variants. The natural assumption is that character 7315 is where a valid body became corrupt. It is not. When a JSON parser is handed a document that contains no value at all, it reports the position at which it gave up looking, which is the end of the input. In our reproduction the body was 7,326 bytes long and consisted entirely of two whitespace byte values, a 100 percent whitespace body. The "char" number was simply its length. The practical rule: any `Expecting value ... (char N)` where N is close to the body length is this failure, not a corruption at offset N. Do not go looking for a damaged byte; there is no content to damage.

3.2 Mode A, budget exhaustion (GLM 5.3, Kimi K3)

Both GLM 5.3 and Kimi K3 default to "max" reasoning effort. On a council-sized review payload at a low ceiling, "max" effort spends the entire budget thinking and leaves nothing for the verdict. The response comes back whitespace. Raising the ceiling alone does not reliably help, because "max" effort simply spends more; the lever is the effort setting, not the ceiling. We confirmed this with a controlled A/B on an identical payload, described in Section 4.1.

3.3 Mode B, the silent window and the sick provider (DeepSeek V4 Pro)

DeepSeek V4 Pro behaves differently and cannot be fixed the same way, because its lowest offered effort is "high"; there is no lower setting to select. It failed in three layers, which we peeled back one at a time, and the honest lesson is that the deepest layer was not the model at all.

The first layer is the silent window. Its high-effort reasoning phase on a council payload is long and near-silent: we measured 13,000 to 19,000 reasoning tokens, corresponding to roughly 50 to 72 seconds of generation during which almost no bytes reach the client. A non-streamed request sitting in that silent window is dropped by an intermediate network hop under parallel load, surfacing as an empty 200 body or an outright receive failure. The second layer is budget: even when the connection survives, at a 16,000-token ceiling the reasoning alone exhausts the ceiling first, and the body returns with a "length" finish reason and no content. Streaming and a larger ceiling address these two.

The third layer only appeared under the true workload, and it is the important one. On the model aggregator, a single model id is served by many backend providers, and the harness pins the seat to a named non-training provider so a result reflects the model rather than an unknown reseller. Our pinned provider began returning an instant "504, operation aborted" on the long-reasoning streamed request, in two consecutive real convenings, while that same provider's published uptime read 99 percent. The published number is an average over all request types; our long silent request is the hard case it hides. Retrying did not help, because every retry landed back on the same degraded provider. The fix at this layer is not in the model or the request at all: it is to widen the provider preference to an ordered list of healthy non-training backends and let the aggregator route to one that is currently serving the hard request, with the retry as a backstop rather than the cure.

4. Experiments

4.0 The harness under test

All runs were fired through the same harness we run in production, so the results describe our real operating conditions rather than a lab rig. It is deliberately small: a single shell script, extracted into one file precisely so that its behaviour stops drifting between convenings, since in our earlier experience every "corruption" incident traced back to a hand-retyped request that quietly dropped a provider pin or a token ceiling. The harness has two verbs. The first, *fire*, takes a model slug, a shared system and user prompt staged to files, and a provider

pin, and issues one independent request; several fires run in parallel as background jobs, which is what makes a nine-seat panel return in the time of its slowest single seat rather than the sum of all nine. The second, *collect*, parses each response and refuses to count a seat whose body carries no verdict.

Three properties of the harness matter for this study. It talks to the models through a single model aggregator (OpenRouter) over one HTTP endpoint, so adding or swapping a seat is a one-line change and the same harness embeds unchanged in any coding agent or pipeline. It pins each seat to a named backend with an explicit provider order, not merely a "no fallbacks" flag, because an aggregator otherwise routes one model id to whichever reseller is cheapest at that moment, and a result from an unknown backend cannot be trusted to reflect the model. And it sanitises control bytes out of every payload, parses each returned body in a real JSON parser rather than a shell text tool, and logs the provider that actually served each request. The native Claude seats in the full council are run separately as in-process subagents rather than through the aggregator; this note concerns only the aggregator-served seats, where the empty-verdict failure occurs. Every experiment below uses this harness, with each seat pinned to a named first-party or non-training provider, and every body verified for a real verdict token before it is counted.

4.1 Controlled A/B: effort cap on GLM 5.3 and Kimi K3

We fired each of GLM 5.3 and Kimi K3 twice on one identical review payload, once at capped "high" effort and once uncapped at the default. Table 2 gives the result. Capped, both returned complete, well-formed verdicts using on the order of 1,300 reasoning tokens. Uncapped, GLM spent over five times as many reasoning tokens on the same task, and the Kimi control reproduced the original failure exactly: a 7,326-byte, entirely whitespace body.

Seat	Effort	Reasoning tokens	Body	Verdict returned
GLM 5.3	high (capped)	1,360	complete	Yes
GLM 5.3	max (default)	7,252	complete on this payload	Yes, but fragile
Kimi K3	high (capped)	1,301	complete	Yes
Kimi K3	max (default)	(exhausted)	7,326 bytes, 100% whitespace	No

Table 2: The effort cap on an identical payload. The capped runs are clean and cheap; the uncapped Kimi control reproduces the empty-body failure, and the uncapped GLM run survives this particular payload only by spending five times the reasoning, which is exactly the margin that vanishes on a larger one.

4.2 Roster health sweep

To separate model-specific failure from harness-wide breakage, we fired the remaining fourteen aggregator seats in one parallel burst, each pinned to its intended provider. Thirteen returned complete verdicts on the first attempt, each served by the provider it was pinned to. The lone failure was DeepSeek V4 Pro, which returned an empty body under the burst, isolating it as a distinct case rather than a general fault.

4.3 DeepSeek instrumentation

We then timed DeepSeek deliberately. Table 3 records the four conditions. A tiny payload returns instantly. The full payload returns correctly when fired in isolation, whether streamed or not, taking 50 to 72 seconds. The same payload fired inside a parallel burst without streaming returns empty. That pattern, works alone, fails under concurrent load, points at the silent-window drop rather than the model.

Condition	Wall time	Result
Tiny payload	~1.4 s	200, correct

Condition	Wall time	Result
Full payload, isolated, non-streamed	~50 s	200, full verdict
Full payload, isolated, streamed	~72 s	200, full verdict
Full payload, parallel burst, non-streamed	varies	empty body (x2)

Table 3: DeepSeek V4 Pro across four conditions. The model is not the fault: it succeeds alone and fails only under concurrent load without streaming, which is the signature of a dropped silent-reasoning window rather than a model error.

4.4 DeepSeek fix, and the failure the first fix hid

We first applied streaming and a 40,000 ceiling and re-ran a parallel burst; it passed, and we nearly reported it fixed. Retesting stopped us. A fresh burst came back short, and the true workload, a mixed convening in which DeepSeek fires once alongside five other models, failed it outright twice: an instant "504, operation aborted" from the pinned provider before reasoning even began, the same 927-byte error body both times. This is the discipline that matters most in this note: a monitor or a fix must be tested against the failure it exists to prevent, under the real workload, not a convenient proxy. The self-parallel burst had been the convenient proxy, and it was measuring the wrong thing.

Querying the aggregator's per-provider endpoint list showed the pinned provider degraded on this specific request while several other non-training backends were healthy on it. We widened the provider preference to a health-ordered list and re-ran the real workload. Table 4 gives the result: across two consecutive mixed convenings, DeepSeek returned a complete verdict both times, served by the healthy backend the aggregator now routed to, with no retry needed. The reasoning counts again exceed the old 16,000 ceiling, so the streaming and headroom fixes remain necessary; the provider re-order is what made them sufficient.

Convening	Provider routed	Finish reason	Reasoning tokens	Verdict
Mixed A (6 seats)	degraded provider	error (504)	none	Empty, before fix
Mixed B (6 seats)	degraded provider	error (504)	none	Empty, before fix
Mixed A (re-ordered pin)	healthy backend	stop	within ceiling	Complete
Mixed B (re-ordered pin)	healthy backend	stop	within ceiling	Complete

Table 4: DeepSeek V4 Pro in the real mixed-convening workload, before and after the provider re-order. Streaming and the 40,000 ceiling were applied throughout; they were necessary but not sufficient until the pin routed away from the degraded provider.

4.5 Review quality recovered

A configured seat is only worth fixing if the review it returns is good. Fired on genuine tasks from our own work, each seat produced a specific, load-bearing finding, not a generic pass. GLM 5.3, reviewing a database access-control trigger, returned an overall reject and identified a denial-of-service vector in which an attacker submitting a victim's phone number would silently block that victim's own later registration; it also noted that revoking execute permission on a trigger function is ineffective, since a trigger fires with the table owner's privilege regardless. Kimi K3, reviewing a code change that replaced a runtime lookup with a build-time list, caught that the code keyed on a mutable title and slug when an immutable identifier was already available, risking suppression of the wrong record. DeepSeek V4 Pro, on the same access-control change, returned approve-with-caution and correctly flagged that the migration was safe in isolation but did not by itself complete the security goal it was part of. These are the kinds of finding that justify an independent panel in the first place.

4.6 Combined verification

With all three seats configured per Section 5, we ran the full workload repeatedly as a final check: each of GLM 5.3, Kimi K3, and DeepSeek V4 Pro fired together, three convenings back to back. Every one of the nine fires returned a complete verdict with a clean stop, each served by its intended provider (GLM by Z.AI, Kimi by Moonshot, DeepSeek routed to the healthy StreamLake backend), and the retry backstop was never invoked. Nine of nine, zero retries, is the evidence behind the claim that the three seats are convene-ready; a single clean run would not have been, which is the whole lesson of Section 4.4.

5. The correct harness setup

The method reduces to six rules. The first five prevent the empty body; the last ensures that if one still occurs, it is never miscounted as agreement.

1. **Pin the provider to a named first-party or non-training backend, with an explicit order.** An aggregator will otherwise route one model id to different backends run by different resellers, and a bad backend is indistinguishable from a bad model until you log which one served the request. Pass an ordered provider list, not merely a "no fallbacks" flag, and log the provider on every response.
2. **Cap reasoning effort where the model allows it.** For a model that defaults to "max" and offers a lower level, select the lower level for review work. This is the whole fix for GLM 5.3 and Kimi K3.
3. **Give reasoning headroom the model cannot turn down.** For a model whose effort floor is already high, capping is not available; instead measure its actual reasoning spend and set the ceiling well above it. We use 40,000 for DeepSeek V4 Pro against a measured floor near 13,000 to 19,000.
4. **Stream any seat with a long silent reasoning phase.** Streaming does not change what the model generates, but it keeps bytes flowing through the reasoning window so an intermediate hop does not drop the request under load. Write the stream to a file and reassemble it; piping the stream directly into a parser can itself trigger a receive failure. When you retry a streamed seat, gate the retry on the reassembled visible content, not on a raw search of the stream text, since the thousands of hidden-reasoning frames will otherwise look like success and the retry will never fire.
5. **Widen the provider preference to a health-ordered list, and re-sort it from live data.** A single provider degrades on the hard request while its average uptime still looks fine, and a retry that lands back on it cannot recover. Pin an ordered list of healthy backends rather than one, re-sort it from the aggregator's per-provider endpoint health when a seat fails a whole convening, and keep every backend inside your data-retention policy. This was the layer that actually fixed DeepSeek; the retry is a backstop, not the cure.
6. **Verify the body, never the status.** Assert that the parsed content contains a real verdict token before counting the seat. An empty or whitespace body is an abstention to be retried or substituted, never a vote. Report the roster honestly as N of M seats, X substituted.

Table 5 gives the per-seat configuration these rules produce for the three models in this study.

Seat	Provider pin	Effort	Ceiling	Stream
GLM 5.3	Z.AI	high (capped)	16,000 to 40,000	No
Kimi K3	Moonshot AI	high (capped)	16,000 to 40,000	No
DeepSeek V4 Pro	StreamLake, CoreWeave, BaseTen, Novita (health-ordered)	high (floor)	40,000	Yes

Table 5: Per-seat configuration. GLM and Kimi are fixed by the effort cap; DeepSeek is fixed by streaming, headroom, and a health-ordered provider list that routes around a degraded backend. The document-heavy ceiling of 40,000 is a safe default for

all three. Provider health is time-varying: the DeepSeek order is a worked example to re-sort from live endpoint data, not a constant.

The following is the core of the harness fire path, with the two model-specific branches. Secrets are placeholders.

```
# Per-model handling before the request is built:
STREAM=""; CEILING=16000; EFFORT="$REQUESTED_EFFORT"
case "$MODEL" in
  *glm-5.3*|*kimi-k3*)    EFFORT="high" ;;          # cap the max-effort default
  *deepseek*)             STREAM=1; CEILING=40000    # stream + headroom (effort floor is high
                        # health-ordered non-training providers; re-sort from live endpoint
                        PIN="StreamLake,CoreWeave,BaseTen,Novita" ;;
esac

# Request body (aggregator schema); pin the provider WITH an order, not just no-fallbacks:
build_body() {
  jq -n --arg m "$MODEL" --argjson mt "$CEILING" --arg re "$EFFORT" \
    --argjson prov "$(jq -n --arg p "$PIN" '{order:($p|split(",")), allow_fallbacks:false}')" \
    --rawfile s system.txt --rawfile u user.txt \
    '{model:$m, max_tokens:$mt, temperature:0.2, provider:$prov,
      messages:[{role:"system",content:$s},{role:"user",content:$u}]}
    + (if $STREAM then {stream:true} else {} end)
    + (if $re != "" then {reasoning:{effort:$re}} else {} end)'
}

# Streamed seats: write raw SSE to a file, THEN reassemble. Do not pipe curl -N into the parse
if [ -n "$STREAM" ]; then
  build_body | curl -s -N --max-time 580 "$ENDPOINT" -H "Authorization: Bearer $KEY" \
    -H "Content-Type: application/json" --data-binary @- -o out.sse
  reassemble_sse out.sse > out.json    # concatenate choices[].delta.content across frames
else
  build_body | curl -s --max-time 280 "$ENDPOINT" -H "Authorization: Bearer $KEY" \
    -H "Content-Type: application/json" --data-binary @- > out.json
fi

# Collect: an empty body is NOT assent.
verdict_present out.json || mark_abstained_and_substitute
```

6. Threats to validity

These results are from one aggregator on one day, against three model versions, fired from one account whose data-retention setting excludes providers that train on request data. That setting is why DeepSeek routes to resellers rather than first-party for us, and another account may route differently. The token ceilings are calibrated to our council payloads, which run to tens of kilobytes; a longer document will move the reasoning floor and the ceiling with it, which is exactly why rule three says to measure rather than copy our number. The failures depend on concurrency and on transient provider load, so a single successful isolated run, and even a passing self-parallel burst, is not evidence that a seat is safe; we learned this the hard way and treat the real mixed convening, the workload as it actually runs, as the only test that counts. Finally, "review quality recovered" is reported from individual tasks, not a scored benchmark; the claim is only that each configured seat returned a specific, verifiable finding, not that any seat outranks another.

7. Related work and conclusion

This note extends *The Intelligence Council* [1], which set out the panel architecture and named this failure without dissecting it. The broader context is the reliability of orchestration layers built on shared model

aggregators [2], where a single model id is a routing target rather than a fixed backend, and where reasoning models have made the request envelope, effort and ceiling and streaming, a first-class operational concern rather than a default one can ignore.

The conclusion is narrow and practical. When a strong model returns nothing, the harness and the route are the first suspects, not the model. Three of our seats failed the same way on the surface and for different reasons underneath, and every reason is configuration or infrastructure, not capability. A reasoning-first model spends from a shared budget, sometimes thinks in a long silence, and is served through a backend that can quietly go bad; a harness that caps effort where it can, gives headroom where it cannot, streams the long-silent seats, routes around a degraded provider, and refuses to count an empty body as a vote, gets full and useful verdicts from all three. We also learned to distrust our own first fix: the DeepSeek configuration looked complete after a passing burst and was not, and only a retest under the real workload found the layer that mattered. That configuration now runs in production for us, and every seat in this study is convene-ready. This is a practitioner experience report.

References

- [1] Tanveer, A. "The Intelligence Council: Provider-Agnostic, Domain-Specialized Multi-Model Review for Autonomous AI Systems." Speedrun AI Labs, SSRN, July 2026.
- [2] OpenRouter. "Model Routing and Provider Selection." Product documentation, openrouter.ai, accessed August 2026.